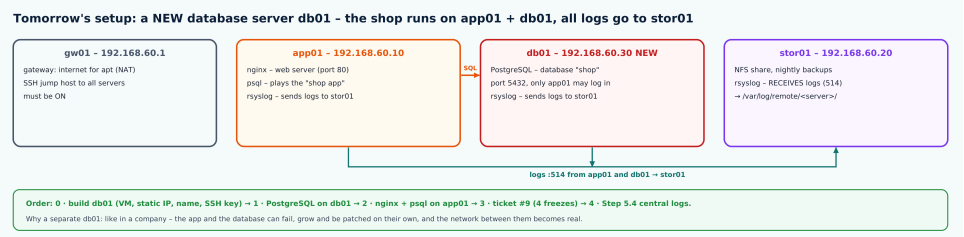
Day 16 – the story: one ticket, a shop on two servers, four places to look

Project 2 – Acme Analytics Linux platform · 9 Nov 2026 · what we built, where each setting lives, and how ticket #9 is solved

GLPI · Ticket #9 · Incident · from Sara (Online shop team) · Sat 07.11 19:31 · item: app01

"I was working in our online shop. At about 19:20 nothing reacted for 3–4 minutes, then it worked again. What happened? Will it happen again?"

The problem for the admin: a web shop can freeze in four places – the app, the whole server, the database, or the network between them. In our lab there was no shop yet, so we **built a small one on two servers** to make all four places real.



1 · What we built, in the order we did it

#	Server	What we did	The main entry / setting	Why
1	**db01** (new VM)	new Ubuntu VM in the lab network	VMware: **Custom → VMnet2**, Connect at power on	VMnet0 = home network (wrong); VMnet2 = the isolated lab network
2	db01	fixed address (VMnet2 has no DHCP)	`/etc/netplan/50-cloud-init.yaml`: `192.168.60.30/24`, `via 192.168.60.1`, DNS `8.8.8.8`	the way out goes **through gw01**
3	**gw01**	**nothing new for routing** – only a name and an SSH key	`/etc/hosts`: `192.168.60.30 db01` · `ssh-copy-id db005@db01`	gw01 already does NAT for the whole 192.168.60.0/24 (Phase 1); db01 just points its default route to gw01
4	**app01**	knows the new name	`/etc/hosts`: `192.168.60.30 db01`	so the shop can say `db01` instead of an address
5	db01	names of the others, SSH server	`/etc/hosts`: gw01, app01, stor01 · `apt install openssh-server`	to reach the others by name; to be managed from gw01
6	db01	installed PostgreSQL 14, made the shop data	user `shopapp`, database `shop`, table `products` (shirt 20, jeans 50, jacket 120)	the shop's data lives here
7	db01	opened the database for app01 only	`postgresql.conf`: `listen_addresses = '*'`, `log_min_duration_statement = 2000` · `pg_hba.conf`: `host shop shopapp 192.168.60.10/32 scram-sha-256`	listen on the network (port 5432); **only app01**, with a password; log queries slower than 2 s
8	**app01**	installed the web server and the database client	`apt install nginx postgresql-client` · `~/.pgpass` (password file, `chmod 600`)	nginx = the shop's web server (port 80); `psql` = plays the "shop app"

2 · How it works now

```
Sara's browser ──HTTP :80──► app01: nginx (web server) + psql ("shop app")
                                  │  SQL query  "SELECT * FROM products;"
                                  └──network :5432──► db01: PostgreSQL
                                       (pg_hba.conf: is it app01? password OK?)
                                  ◄── 3 rows back: shirt, jeans, jacket
db01 ──default route──► gw01 (NAT) ──► internet   (only for apt / updates)
```

Proven today: nginx answers **200 OK** on app01, and `psql -h db01 -U shopapp -d shop -c "SELECT * FROM products;"` on app01 returns the **3 rows from db01**. The first try said *password authentication failed* – read left to right: the network worked, the access rule worked, only the password was mistyped.

3 · Ticket #9 – the four places, where to look, and what we found

Case	Where it breaks	Command that answers it	What it shows	Status
① the app	nginx on app01 stopped / hung	`curl -I http://localhost` `journalctl -u nginx --since "10 min ago"`	"Connection refused"; nginx's own log: Stopping → Stopped → Starting → Started with the times (e.g. down 21:01:03 – 21:01:24)	✔ **done**
② the whole server	app01 itself froze (no CPU, no RAM)	`journalctl --list-boots` `journalctl -k -p err -b -3`	the run Sat 15:42–20:37 (boot -3) has **"soft lockup – CPU#1 stuck"** (18:06, 25 s) and at 19:22 "stuck for 160s", "Watchdog timeout", "Reset adapter"	✔ **found**
③ the database	PostgreSQL on db01 answers very late	app01: `psql -h db01 … -c "SELECT pg_sleep(10);"` db01: `sudo tail /var/log/postgresql/postgresql-14-main.log`	app01 waited 10.28 s; log: "duration: 10088.560 ms statement: SELECT pg_sleep(10)" – the slow-query log names the slow query	✔ **done**
④ the network app01 ⇄ db01	the connection on port 5432 drops	db01: `iptables -I INPUT -s 192.168.60.10 -p tcp --dport 5432 -j DROP` app01: `PGCONNECT_TIMEOUT=5 psql …`, `ping db01` db01: same rule with `-D` to remove it	"timeout expired" while ping had 0 % loss – only port 5432 was blocked; after -D the 3 rows came back	✔ **done**

Lesson from ②: boot numbers move back after every restart – Saturday's run was `-b -1` on Saturday, `-3` today. Always check `journalctl --list-boots` first.

GLPI · Ticket #9 · Solution

"Hi Sara, the shop itself and the database were fine. At 19:22 the whole server app01 got no processor time for about 2–3 minutes – the kernel log shows 'CPU stuck for 160s' and 'Watchdog timeout'. The cause was outside app01: the virtualization host was not giving it CPU (in our lab: the host PC went to sleep). It will not happen again as long as the host stays awake; in a company this goes to the virtualization team as an overloaded host."

Learned on the way: a test run on db01 itself was refused ("no pg_hba.conf entry for host 192.168.60.30") – the rule allows only app01 · "password retrieved from file .pgpass … failed" = the password file and the database disagreed – set the same password in both · **never publish** screenshots that show the database password · **Next:** Step 5.4 – app01 and db01 send their logs to stor01 (add `db01` to stor01's /etc/hosts first).

Final paragraph: Ticket #9 said the web shop froze for 3–4 minutes, which can come from the app, the whole server, the database or the network between them. To make all four real we built a small shop on two servers: a new server db01 in the lab network VMnet2 with the fixed address 192.168.60.30 (netplan, because VMnet2 has no DHCP), whose way to the internet goes through gw01 – gw01 needed no new routing, because it already does NAT for the whole lab network; it only got the name db01 and its SSH key. app01 and gw01 got the name db01 in /etc/hosts, and db01 got the names of the others. On db01, PostgreSQL 14 holds the shop database with a products table, listens on port 5432 and lets only app01 log in with a password, and logs every query slower than 2 seconds. On app01, nginx is the web server and psql plays the shop app; a real SQL query from app01 to db01 returns the three products. For the ticket, each place has its own log: nginx's log answered case ① (stopped and started with the times), the kernel log of Saturday's run answered case ② (CPU stuck, watchdog timeout – the real cause), the database's slow-query log on db01 answered case ③ (duration 10088 ms for a 10-second query), and a blocked port 5432 answered case ④ ("timeout expired" while ping still worked) – so all four places were checked and ticket #9 is solved.